Runbook de Incidentes — Kumi TTPN

Fecha: 2026-04-11
Audiencia: CTO, Tech Lead, dev de guardia
Uso: Consultar cuando algo falla en producción. Ir directo a la sección del síntoma.

Índice de Escenarios

#	Síntoma	Severidad
1	API no responde / Error 502 o 503	🔴 Crítico
2	Error 500 en cualquier endpoint	🔴 Crítico
3	Login falla sin razón aparente	🔴 Crítico
4	Nómina o facturación atascada	🔴 Crítico
5	BD no responde o conexiones agotadas	🔴 Crítico
6	El cuadre automático no hace match	🟠 Alto
7	Viajes sin `payroll_id`	🟠 Alto
8	Servidor lento o reiniciándose solo	🟠 Alto
9	Cron jobs no corrieron	🟡 Medio
10	FE carga pero todas las llamadas dan 401	🟡 Medio
11	Alertas no se generan	🟡 Medio
12	Importación de bookings falla	🟡 Medio
13	FE no carga o muestra versión vieja	🟡 Medio
14	Migración rompió algo, hay que revertir	🟠 Alto

Accesos Rápidos

```
# Railway — ver logs del API en vivo
railway logs --service api -f

# Supabase — consola SQL
# https://app.supabase.com → proyecto TTPN → SQL Editor

# Sidekiq Web UI (solo con cuenta sadmin)
# https://tu-dominio.railway.app/sidekiq

# Ver jobs en cola (desde Rails console en Railway)
railway run bundle exec rails console
```

```
> Sidekiq::Queue.all.map { |q| "#{q.name}: #{q.size}" }
> Sidekiq::RetrySet.new.size    # jobs fallidos en retry
> Sidekiq::DeadSet.new.size     # jobs muertos definitivamente
```

---

1. El API no responde / Error 502 o 503

Síntoma: El FE muestra "Error de red" o "No se pudo conectar". Endpoints devuelven 502 o el browser no recibe respuesta.

Diagnóstico

```
# 1. Verificar que el servicio está corriendo en Railway
# Railway Dashboard → proyecto → service "api" → ver status

# 2. Health check manual
curl https://TU_DOMINIO.railway.app/up
# Debe responder: "API TTPN Online" con 200

# 3. Ver logs recientes
railway logs --service api --lines 100
```

Causas comunes y solución

Causa A — El proceso de Puma murió:

```
# Railway reinicia automáticamente por Restart Policy
# Si no reinicia en 2 minutos: Railway Dashboard → service → Restart
# Verificar en logs: "Puma starting" indica que levantó correctamente
```

Causa B — Memoria agotada (OOM Kill):

```
# En logs verás: "Worker 1234 exceeds memory limit" o "Killed"
# puma_worker_killer debería haberlo manejado, pero si mató el proceso
principal:
# → Railway Dashboard → service api → Restart
# Monitorear memoria post-reinicio: Railway Metrics
```

Causa C — Migración pendiente bloquea el boot:

```
# En logs verás: "ActiveRecord::PendingMigrationError"
railway run bundle exec rails db:migrate
# Luego reiniciar el servicio
```

Causa D — Variable de entorno faltante:

```
# En logs verás: KeyError o nil references en boot
# Railway Dashboard → service api → Variables
# Verificar: DATABASE_URL, SECRET_KEY_BASE, REDIS_URL, RAILS_ENV
```

Escalación

Si el servicio no levanta después de 10 minutos → revisar Supabase status y Railway status page.

---

2. Error 500 en cualquier endpoint

Síntoma: Las llamadas retornan `{"error": "..."}` con status 500, o el FE muestra errores genéricos.

Diagnóstico

```
# Ver el error exacto en logs
railway logs --service api -f
# Buscar líneas que contengan "ERROR" o el stack trace

# Reproducir localmente si es urgente
docker compose exec api bundle exec rails console
```

Causas comunes

Causa A — Migración no ejecutada (la más frecuente):

```
Síntoma en logs: ActiveRecord::PendingMigrationError: ...
Solución:
  railway run bundle exec rails db:migrate
  # O en Railway: Deploy → Run Command → "bundle exec rails db:migrate"
```

Causa B — Error en un modelo o callback:

```
Síntoma en logs: NoMethodError, NameError, undefined method...
Solución: identificar la línea del stack trace, hacer fix, nuevo deploy
```

Causa C — Función PG faltante o con error:

```
Síntoma en logs: PG::UndefinedFunction: ERROR: function buscar_nomina()
does not exist
Solución: verificar en Supabase SQL Editor que la función existe:
```

```
  SELECT routine_name FROM information_schema.routines
  WHERE routine_schema = 'public' AND routine_type = 'FUNCTION'
  ORDER BY routine_name;
Si falta: correr la migración que la crea
```

Causa D — Redis no disponible (Sidekiq jobs sincrónicos que llaman a Redis):

```
Síntoma: Redis::CannotConnectError
Solución: Railway Dashboard → service redis → verificar que está corriendo
```

---

3. Login falla — "Credenciales inválidas" en usuarios válidos

Síntoma: Un usuario con credenciales correctas no puede entrar. El error es `{"error": "Credenciales inválidas"}`.

Diagnóstico

```
railway run bundle exec rails console
> u = User.find_by(email: "usuario@ttpn.com.mx")
> u.nil?              # si true → el usuario no existe en prod
> u.is_active         # si false → está desactivado
> u.valid_password?("su_password")  # true/false
```

Causas comunes

Causa A — Usuario desactivado:

```
# En Rails console de producción:
u = User.find_by(email: "email@ttpn.com.mx")
u.update(is_active: true)
```

Causa B — BD apunta a ambiente equivocado:

```
Verificar en Railway: variable DATABASE_URL apunta a Supabase PRODUCCIÓN
No al proyecto de desarrollo local
```

Causa C — JTI corrupto (token revocado inesperadamente):

```
# Si el usuario puede hacer login pero inmediatamente da 401:
u = User.find_by(email: "email@ttpn.com.mx")
```

```
u.update_column(:jti, SecureRandom.uuid)  # regenera el JTI
# El usuario tendrá que volver a iniciar sesión
```

Causa D — Migración pendiente que agregó campo nuevo:

```
Síntoma: ActiveRecord::StatementInvalid en el login
Solución: db:migrate
```

---

4. Sidekiq no procesa jobs — Nómina o Facturación atascada

Síntoma: El usuario inicia el cálculo de nómina o facturación y el progreso se queda en 0% o en un porcentaje sin avanzar.

Diagnóstico

```
# 1. Verificar que Sidekiq está corriendo
railway logs --service sidekiq --lines 50

# 2. Ver estado de colas desde la web UI
# https://TU_DOMINIO.railway.app/sidekiq  (requiere login sadmin)

# 3. Desde Rails console
railway run bundle exec rails console
> Sidekiq::Queue.all.map { |q| [q.name, q.size] }
> Sidekiq::RetrySet.new.size    # jobs en retry
> Sidekiq::DeadSet.new.size     # jobs muertos
```

Causas comunes

Causa A — Sidekiq no está corriendo:

```
# Railway Dashboard → service sidekiq → Restart
# Verificar en logs: "Sidekiq X.X.X connecting to Redis"
```

Causa B — Redis no disponible:

```
# Sin Redis, Sidekiq no puede conectarse ni procesar nada
# Railway Dashboard → service redis → verificar status
# Si Redis reinició, los jobs en memoria se perdieron
# → El usuario debe reiniciar el cálculo desde el FE
```

Causa C — Job murió con error (está en RetrySet o DeadSet):

```
# Ver el error del job fallido:
Sidekiq::RetrySet.new.first.item   # ver qué falló
Sidekiq::DeadSet.new.first.item

# Relanzar el job desde la web UI:
# /sidekiq → Retries → botón "Retry All"
# O limpiar jobs muertos: /sidekiq → Dead → "Delete All"
```

Causa D — Nómina con `processing_status: 'processing'` huérfano:

```
# Si el proceso murió a mitad, la nómina quedó atascada en 'processing':
p = Payroll.find(ID_NOMINA)
p.update(processing_status: 'pending', progress: 0)
# Luego el usuario puede reiniciar el cálculo desde el FE
```

Causa E — Job de nómina en cola `payrolls` con job en cola `default`:

```
# Las colas son: default, payrolls, alerts
# Verificar que el sidekiq.yml incluye las tres colas
```

---

5. Base de Datos no responde / Conexiones agotadas

Síntoma: Errores `PG::ConnectionBad`, `ActiveRecord::ConnectionTimeoutError`, o queries que no responden.

Diagnóstico

```
# 1. Verificar Supabase status
# https://status.supabase.com

# 2. Ver conexiones activas en Supabase
# Supabase Dashboard → Database → Roles → ver conexiones activas

# 3. Desde SQL Editor en Supabase:
SELECT count(*), state FROM pg_stat_activity GROUP BY state;
SELECT pid, now() - query_start AS duration, query, state
FROM pg_stat_activity
WHERE state != 'idle'
ORDER BY duration DESC
LIMIT 20;
```

Causas comunes

Causa A — Conexiones agotadas (pool lleno):

```
-- Terminar conexiones idle en Supabase SQL Editor:
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE state = 'idle'
  AND query_start < now() - interval '10 minutes';
```

```
# En Rails: ajustar pool size en database.yml si es recurrente
# Supabase Pro soporta hasta 500 conexiones directas
# Con PgBouncer (incluido en Supabase): hasta 10,000 conexiones lógicas
```

Causa B — Query lenta bloqueando todo:

```
-- Identificar y matar la query bloqueante:
SELECT pid, now() - query_start AS duration, query
FROM pg_stat_activity
WHERE state != 'idle'
ORDER BY duration DESC;

SELECT pg_cancel_backend(PID_DEL_BLOCKING_QUERY);
-- Si no responde al cancel:
SELECT pg_terminate_backend(PID_DEL_BLOCKING_QUERY);
```

Causa C — Supabase en mantenimiento programado:

```
Supabase Pro avisa con 7 días de anticipación.
Durante el mantenimiento (~5-10 min), la BD no está disponible.
→ El FE mostrará error de conexión.
→ No hay acción requerida. Esperar a que termine.
→ Verificar en https://status.supabase.com
```

Causa D — Trigger PG con error que bloquea inserts:

```
-- Si los inserts de travel_counts fallan:
-- Verificar que las funciones del trigger existen:
SELECT routine_name FROM information_schema.routines
WHERE routine_schema = 'public' AND routine_type = 'FUNCTION';
-- Deben estar: buscar_booking, buscar_booking_id, buscar_nomina,
sp_tctb_insert, sp_tctb_update
```

---

6. El cuadre automático dejó de funcionar

Síntoma: Los TravelCount se crean pero `viaje_encontrado` siempre queda en `false`, aunque hay bookings que deberían hacer match.

Diagnóstico

```
-- En Supabase SQL Editor, verificar que los triggers existen:
SELECT trigger_name, event_object_table, event_manipulation
FROM information_schema.triggers
WHERE trigger_schema = 'public'
ORDER BY event_object_table;
-- Deben aparecer: tc_insert, tc_update, sp_tctb_insert, sp_tctb_update,
sp_tb_update

-- Probar la función de cuadre manualmente:
SELECT buscar_booking(
  vehicle_id,       -- integer
  employee_id,      -- integer
  service_type_id,  -- integer
  foreign_destiny_id, -- integer
  client_id,        -- integer
  '2026-04-11',     -- date
  '08:00:00'        -- time
);
```

Causas comunes

Causa A — Trigger eliminado por una migración:

```
# Verificar qué migraciones se corrieron recientemente:
railway run bundle exec rails runner "
ActiveRecord::SchemaMigration.order(ran_at: :desc).limit(5).each { |m|
puts m.version }
"
# Si una migración reciente tocó travel_counts, puede haber eliminado el
trigger
# Correr la migración que recrea los triggers
```

Causa B — Clv_servicio no coincide entre TravelCount y TtpnBooking:

```
-- Comparar las claves en registros que deberían cuadrar:
SELECT id, clv_servicio FROM ttpn_bookings
WHERE fecha = '2026-04-11' AND viaje_encontrado = false
LIMIT 10;

SELECT id, employee_id, vehicle_id, client_branch_office_id,
       ttpn_service_type_id, fecha, hora
```

```
FROM travel_counts
WHERE fecha = '2026-04-11' AND viaje_encontrado = false
LIMIT 10;
-- Verificar que los IDs de employee, vehicle, client coincidan
```

Causa C — `buscar_nomina()` retorna NULL (nómina no configurada):

```
SELECT buscar_nomina();
-- Si retorna NULL: no hay nómina activa para la fecha actual
-- Crear una nómina desde el FE que cubra el período actual
```

---

7. Viajes sin `payroll_id` — Nómina incompleta

Síntoma: Al generar la nómina, faltan viajes. Los TravelCount existen pero no tienen `payroll_id`.

Diagnóstico

```
-- Cuántos travel_counts sin payroll_id existen:
SELECT count(*) FROM travel_counts
WHERE payroll_id IS NULL AND created_at > '2026-01-01';

-- Ver el rango de fechas afectado:
SELECT min(fecha), max(fecha) FROM travel_counts WHERE payroll_id IS NULL;
```

Causas comunes

Causa A — No había nómina activa cuando se insertaron los viajes:

```
-- Verificar que buscar_nomina() cubre el período:
SELECT * FROM payrolls
WHERE fecha_inicio <= CURRENT_DATE AND fecha_hasta >= CURRENT_DATE;
-- Si no hay registro: crear una nómina desde el FE

-- Asignar retroactivamente el payroll_id correcto:
UPDATE travel_counts
SET payroll_id = (SELECT id FROM payrolls
                  WHERE fecha_inicio <= travel_counts.fecha
                    AND fecha_hasta >= travel_counts.fecha
                  LIMIT 1)
WHERE payroll_id IS NULL
  AND fecha BETWEEN '2026-03-01' AND '2026-03-31';
-- IMPORTANTE: verificar el UPDATE con SELECT antes de ejecutarlo
```

Causa B — El trigger `sp_tctb_insert` no llama a `buscar_nomina()`:

```
-- Verificar que el trigger tiene la llamada:
SELECT routine_definition FROM information_schema.routines
WHERE routine_name = 'sp_tctb_insert';
-- Buscar en el texto: "buscar_nomina"
-- Si no está: correr la migración 20260410000002
```

---

8. Alto consumo de memoria — Servidor lento

Síntoma: Las respuestas tardan más de 5 segundos, Railway Metrics muestra memoria alta, o aparecen en logs mensajes de `puma_worker_killer`.

Diagnóstico

```
# Ver métricas en Railway Dashboard → service api → Metrics
# Referencia: consumo normal ~200-350MB, crítico >450MB

# Ver en logs si puma_worker_killer ya actuó:
railway logs --service api | grep -i "worker\|memory\|killed"
```

Acciones

Acción 1 — Reinicio suave (no hay downtime):

```
# Railway hace rolling restart automático si el health check falla
# Para forzarlo manualmente:
# Railway Dashboard → service api → Deploy → Restart
```

Acción 2 — Identificar la fuente de la fuga:

```
# Desde rails console en Railway:
railway run bundle exec rails console
> GC.stat[:heap_live_slots]       # objetos vivos en heap
> ObjectSpace.count_objects        # distribución por tipo

# Jobs que consumen más memoria:
# PayrollCalculationJob — procesa miles de travel_counts
# TtpnBookingImportJob — importa Excel con muchos registros
# → Si coincide con el pico, ajustar batch size en el job
```

Acción 3 — Ajuste temporal de `puma_worker_killer`:

```
```

```
# config/initializers/puma_worker_killer.rb
# Bajar el umbral si los reinicios no son suficientemente frecuentes:
PumaWorkerKiller.config { |c|
  c.ram           = 400  # MB (bajar de 512)
  c.percent_usage = 0.95
  c.rolling_restart_frequency = 3600  # reinicio preventivo cada hora
}
```

---

9. Jobs de Cron no corrieron

Síntoma: No se desactivaron versiones expiradas, o no se corrió la verificación de documentos vencidos.

Jobs programados

Job	Cron	Qué hace
`DeactivateExpiredVersionsJob`	`0 2 * * *` (2am)	Desactiva versiones de app con `fecha_fin < hoy`
`DocExpirationCheckJob`	`0 6 * * *` (6am)	Revisa documentos de empleados/vehículos próximos a vencer

Diagnóstico

```
# 1. Verificar que Sidekiq y sidekiq-cron están corriendo:
railway logs --service sidekiq | grep -i "cron\|schedule"

# 2. Ver desde consola si los jobs están registrados:
railway run bundle exec rails console
> Sidekiq::Cron::Job.all.map { |j| [j.name, j.cron, j.last_enqueue_time] }
```

Causas comunes

Causa A — Sidekiq se reinició y perdió el schedule:

```
# El schedule se carga al iniciar Sidekiq desde sidekiq.yml
# Si Sidekiq reinició sin releer el schedule:
# Railway Dashboard → service sidekiq → Restart
# Al iniciar, sidekiq-cron lee el schedule automáticamente
```

Causa B — Job falló y está en RetrySet:

```
railway run bundle exec rails console
> Sidekiq::RetrySet.new.select { |j| j.item['class'].include?
```

```
('Deactivate') }
# Si está en retry → /sidekiq → Retries → Retry Now
```

Causa C — Correr el job manualmente (parche inmediato):

```
railway run bundle exec rails console
> DeactivateExpiredVersionsJob.perform_now
> DocExpirationCheckJob.perform_now
```

---

10. Frontend carga pero API retorna 401 en todas las llamadas

Síntoma: El FE carga correctamente pero todas las llamadas al API retornan 401 "Sesión expirada" aunque el usuario acaba de hacer login.

Diagnóstico

```
1. Abrir DevTools → Network → ver la llamada fallida
2. Verificar que el header Authorization: Bearer <token> está presente
3. Copiar el token y decodificarlo en jwt.io para ver el contenido
```

Causas comunes

Causa A — SECRET_KEY_BASE cambió en Railway:

```
Si la variable SECRET_KEY_BASE cambia, todos los JWTs existentes son
inválidos.
→ Todos los usuarios deben hacer login de nuevo.
→ Verificar que SECRET_KEY_BASE es la misma que antes del deploy.
```

Causa B — El token expiró (exp: 24 horas):

```
El token dura 24 horas. Si el usuario no refrescó la página en ese tiempo:
→ Hacer logout desde el FE y volver a entrar.
→ Evaluar si el FE maneja refresh token automático.
```

Causa C — JTI revocado inesperadamente:

```
# Si el user.jti cambió (por un deploy que corrió algún seed o migración):
railway run bundle exec rails console
> u = User.find_by(email: "email@ttpn.com.mx")
```

```
> u.update_column(:jti, SecureRandom.uuid)
# El usuario hace logout + login y obtiene un token con el nuevo JTI
```

Causa D — CORS bloqueando el header Authorization:

```
Síntoma: funciona desde curl pero no desde el browser
Revisar: config/initializers/cors.rb
Verificar que el dominio de Netlify está en la lista de origins permitidos
```

---

11. Alertas no se están disparando

Síntoma: Hay discrepancias, viajes sin cuadrar, o documentos vencidos pero no aparecen alertas en la campana.

Diagnóstico

```
railway run bundle exec rails console
> Alert.where(status: 'pending').count   # cuántas hay
> AlertRule.where(active: true).count    # cuántas reglas activas
> Sidekiq::Queue.new('alerts').size      # jobs de alerta pendientes
```

Causas comunes

Causa A — La cola `alerts` de Sidekiq no existe o no se procesa:

```
# Verificar en sidekiq.yml que la cola alerts está configurada:
# queues: [default, payrolls, alerts]
# Railway Dashboard → service sidekiq → Restart si se cambió el yml
```

Causa B — `AlertDispatchJob` falla silenciosamente:

```
# Ver jobs muertos relacionados con alertas:
Sidekiq::DeadSet.new.select { |j| j.item['class'] == 'AlertDispatchJob' }
# Ver el error y corregirlo
```

Causa C — No hay reglas de alerta configuradas:

```
FE → Settings → Integraciones → Reglas de Alertas
Verificar que hay reglas activas para los eventos que se esperan.
```

---

12. Error en importación de Bookings Excel

Síntoma: El usuario sube el Excel de bookings y el proceso falla o importa registros incorrectos.

Diagnóstico

```
railway run bundle exec rails console
> Sidekiq::RetrySet.new.select { |j| j.item['class'] ==
'TtpnBookingImportJob' }
# Ver el error específico del job
```

Causas comunes

Causa A — Formato de columnas incorrecto:

```
El Excel debe seguir el formato documentado en:
ttpn-frontend/documentacion/FORMATO_EXCEL_IMPORTACION.md
→ Revisar que las columnas coincidan exactamente con el template
```

Causa B — Registros con datos faltantes (cliente o servicio que no existe):

```
# El job retorna los IDs de filas con error
# Ver en Sidekiq RetrySet el mensaje de error que incluye fila y campo
# Corregir el Excel y volver a importar
```

Causa C — Job atascado en `processing`:

```
# Si el import quedó a mitad:
# No hay estado en BD para imports — el job es idempotente
# → Volver a subir el mismo Excel, el job procesará las filas restantes
```

---

13. Netlify — Frontend no carga o muestra versión vieja

Síntoma: El FE muestra una pantalla en blanco, error de build, o una versión antigua del código.

Diagnóstico

```
# Ver status del deploy en Netlify Dashboard
# https://app.netlify.com → tu sitio → Deploys
```

Causas comunes

Causa A — Build fallido:

```
Netlify Dashboard → Deploys → ver el deploy fallido → leer el log
Causa más frecuente: variable de entorno faltante o error de sintaxis en
Vue
Solución: corregir el error y hacer push → Netlify redeploya
automáticamente
```

Causa B — Cache del browser / Service Worker desactualizado:

```
El usuario ve versión vieja en su browser:
→ Ctrl+Shift+R (hard refresh)
→ O en DevTools → Application → Service Workers → Unregister → recargar
```

Causa C — Variable `API_URL` incorrecta en Netlify:

```
Netlify Dashboard → Site Settings → Environment Variables
Verificar que API_URL apunta al dominio correcto de Railway (producción)
Después de cambiar: Site Settings → Deploys → Trigger Deploy
```

---

14. Rollback de Migración de Base de Datos

Síntoma: Se corrió una migración en producción y rompió algo. Hay que revertirla.

⚠ Advertencia

Las migraciones que tocan triggers PG (`sp_tctb_insert`, funciones de cuadre) son las más delicadas. Un rollback incorrecto puede dejar el cuadre sin funcionar.

Procedimiento

```
# 1. Identificar la migración que falló
railway run bundle exec rails runner "
ActiveRecord::SchemaMigration.order(ran_at: :desc).limit(3).each { |m|
puts m.version + ' - ' + m.ran_at.to_s }
"

# 2. Hacer rollback de la última migración
railway run bundle exec rails db:rollback

# 3. Para rollback de N pasos:
railway run bundle exec rails db:rollback STEP=2

# 4. Verificar que el sistema funciona:
```

```
curl https://TU_DOMINIO.railway.app/up
# Hacer login manual en el FE
# Crear un travel_count de prueba y verificar que el cuadre funciona
```

Si el rollback también falla

```
-- Opción nuclear: restaurar desde backup de Supabase
-- Supabase Pro hace backup automático diario
-- Supabase Dashboard → Database → Backups → Restore

-- IMPORTANTE: restaurar un backup restaura TODOS los datos al punto
anterior
-- Esto borra cualquier dato registrado desde el backup hasta ahora
-- Solo usar en casos extremos — coordinarlo con el equipo operativo
```

Contactos y Escalación

Nivel	Cuándo escalar	Acción
Dev de guardia	Cualquier incidente	Revisar este runbook primero
Tech Lead	Incidente persiste >30 min o afecta operación de choferes	Notificar directamente
CTO	Pérdida de datos, BD comprometida, incidente >2 horas	Escalar inmediatamente
Soporte Railway	API/Sidekiq no levantan y no es error de código	https://railway.app/help
Soporte Supabase	BD no responde y status.supabase.com muestra incidente	Panel de soporte Supabase Pro

---

Checklist Post-Incidente

Después de resolver cualquier incidente 🔴 o 🟠 :

- ☐ Confirmar que el sistema funciona (health check + login manual)
- ☐ Verificar que los viajes del período no se perdieron (`TravelCount.where(fecha: Date.today).count`)
- ☐ Verificar que Sidekiq está procesando (`Sidekiq::Queue.all.map { |q| [q.name, q.size] }`)
- ☐ Verificar que los triggers de cuadre funcionan (crear un travel_count de prueba)
- ☐ Documentar qué pasó, cuánto duró y cómo se resolvió (en Notion, Slack o donde se lleve el historial)
- ☐ Si fue por código: abrir issue/ticket para el fix definitivo